

Modern Reinforcement Learning: From Agents to LLMs

Post-Training LLMs: KL-Regularized Policy Optimization
SFT → PPO → DPO → GRPO

Lecture 8

Keywords: LLM as policy, SFT, Preferences, Policy Optimization, PPO, DPO, GRPO

Ilija Bogunovic

Eval form

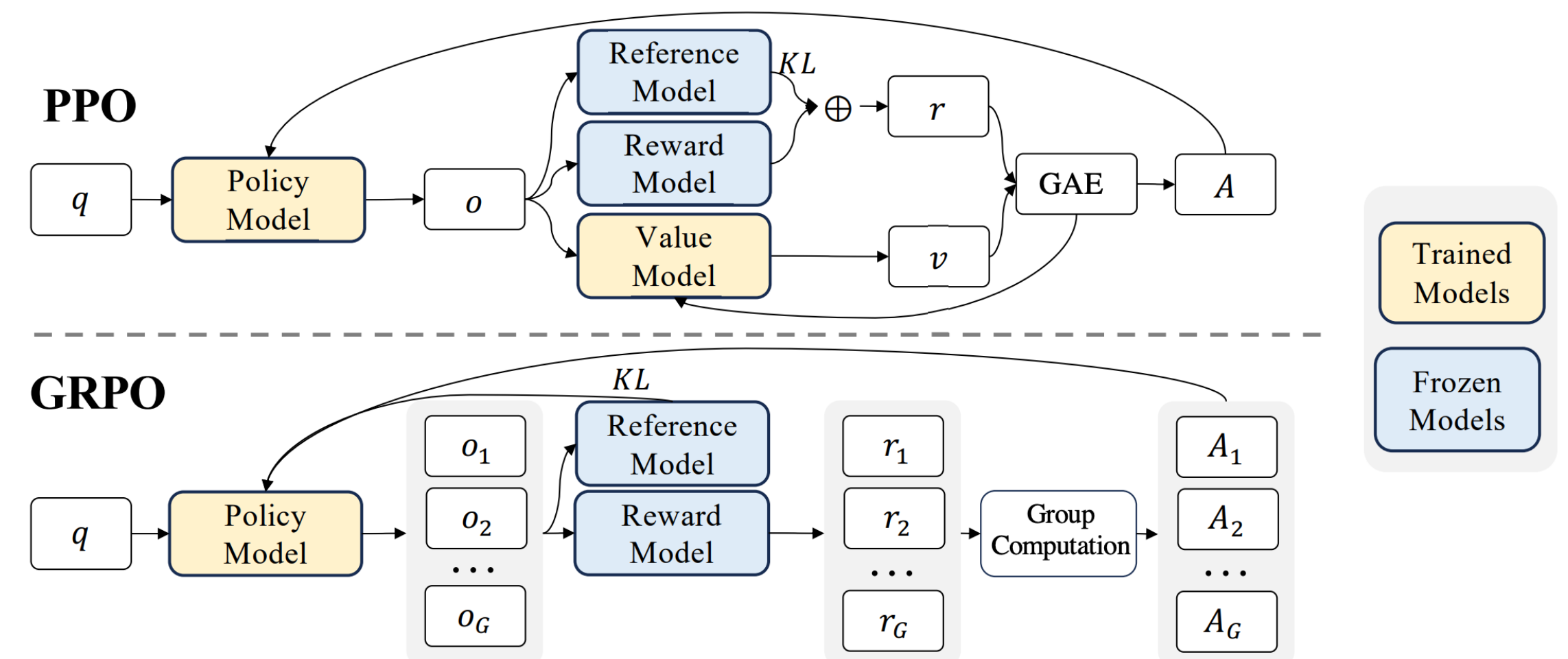
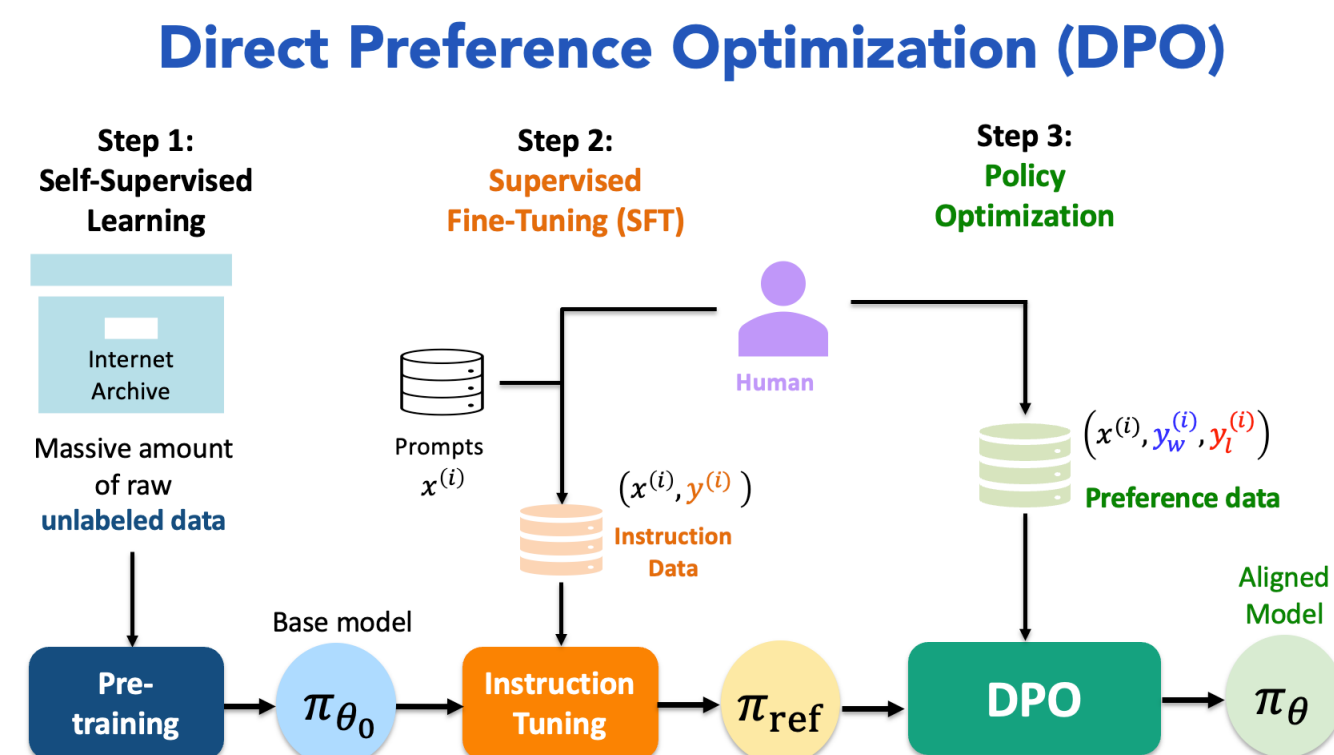
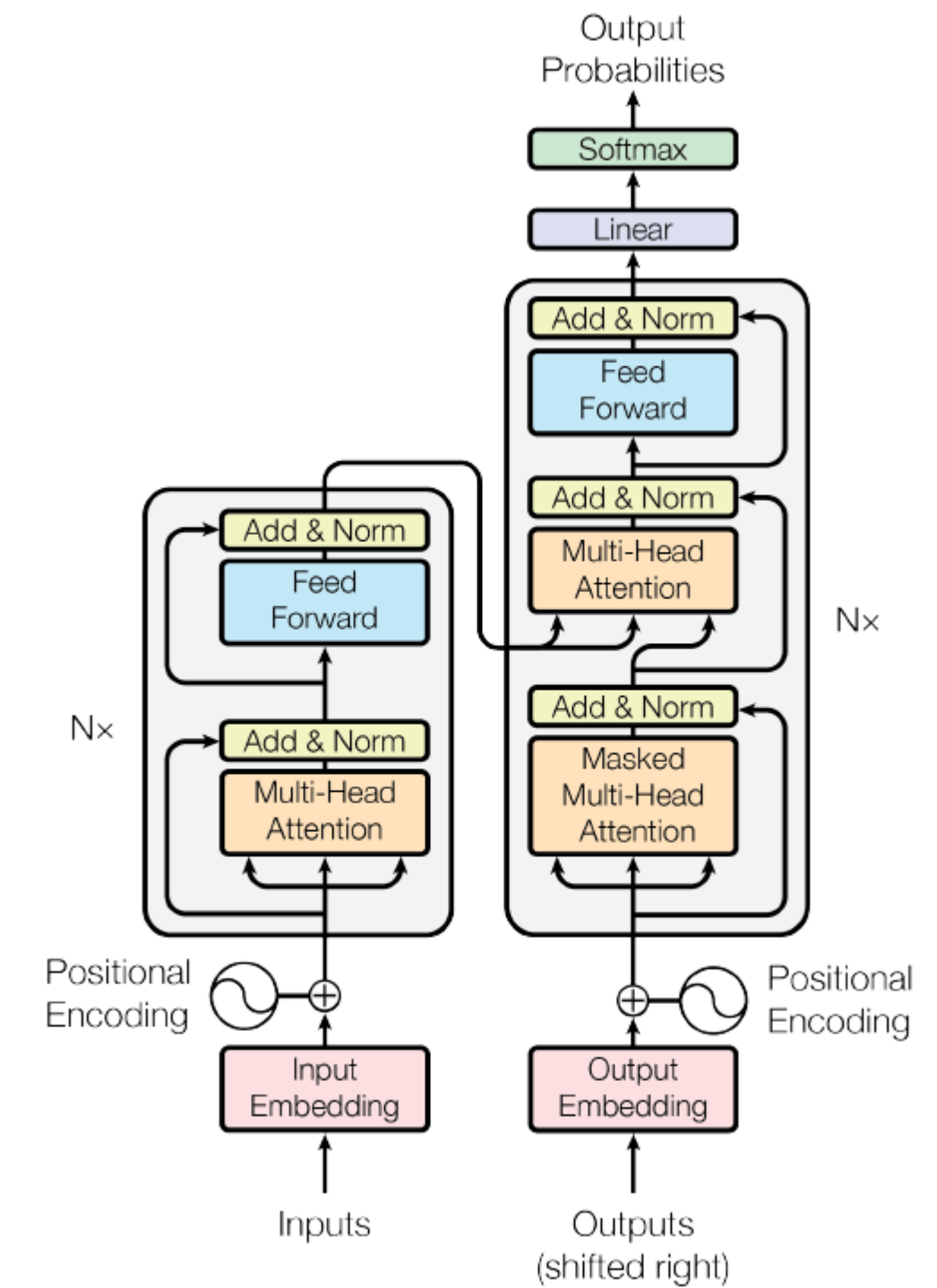
<https://evasys.unibas.ch/evasys/online.php?pswd=SZUTZ>



Lecture A

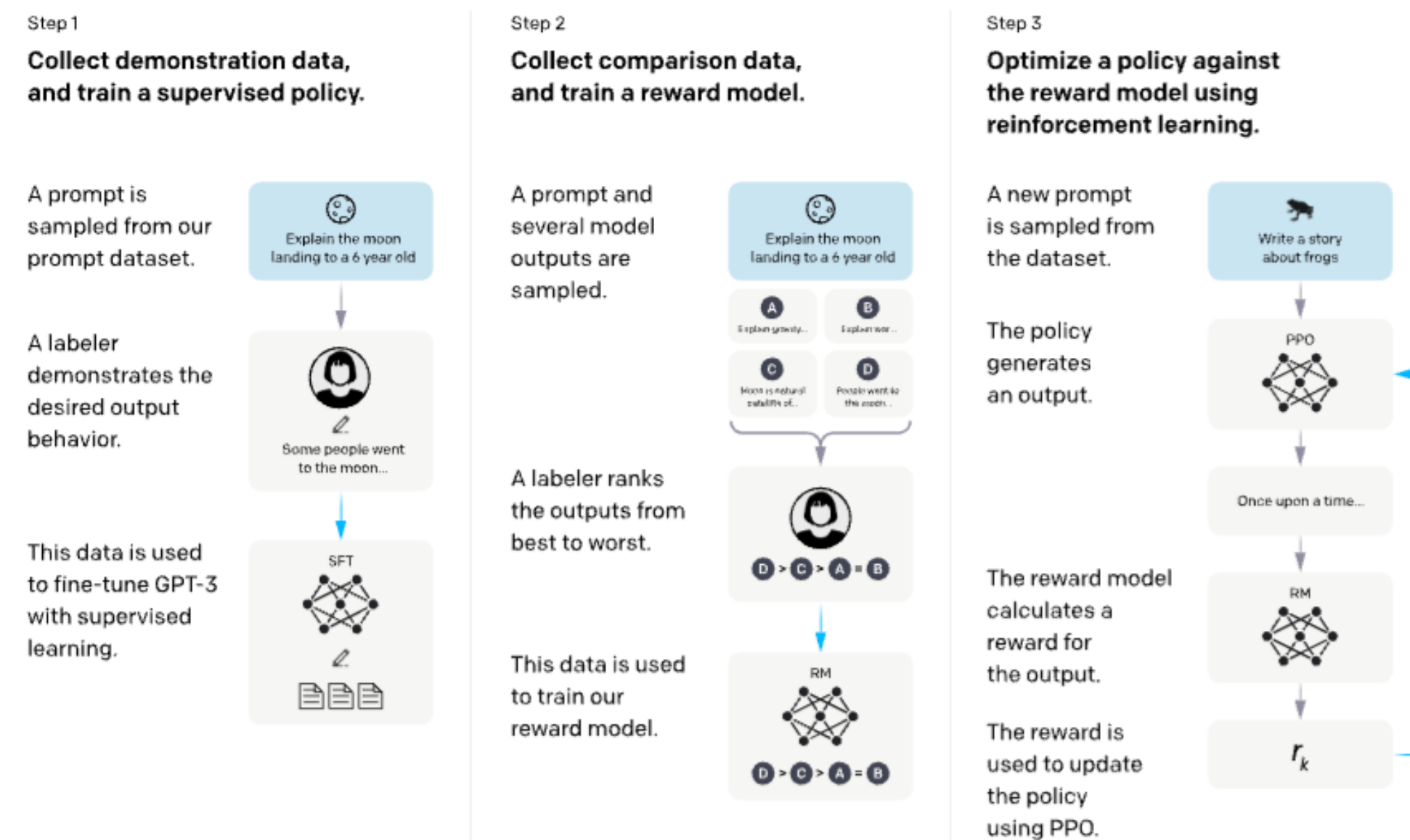
Today's Roadmap

1. Transformer → policy view (attention + causal mask)
2. SFT objective and why it defines π_{ref}
3. KL-regularized post-training objective
4. Closed-form optimum π^*
5. PPO → DPO → GRPO (connections)
6. Demos



Course Map: From Preferences to Policy Optimization

Pretraining → SFT → Preference data (BT) → RM or Direct → KL-regularized optimization



- Lecture 7: preferences + Bradley–Terry + reward models
- Lecture 8A: SFT + KL-regularized objective
- Lecture 8B: PPO / DPO / GRPO + demos

LLM As a Stochastic Policy

- Prompt x , completion $y = (y_1, \dots, y_T)$
- Autoregressive policy factorization:

$$\pi_{\theta}(y \mid x) = \prod_{t=1}^T \pi_{\theta}(y_t \mid x, y_{<t})$$

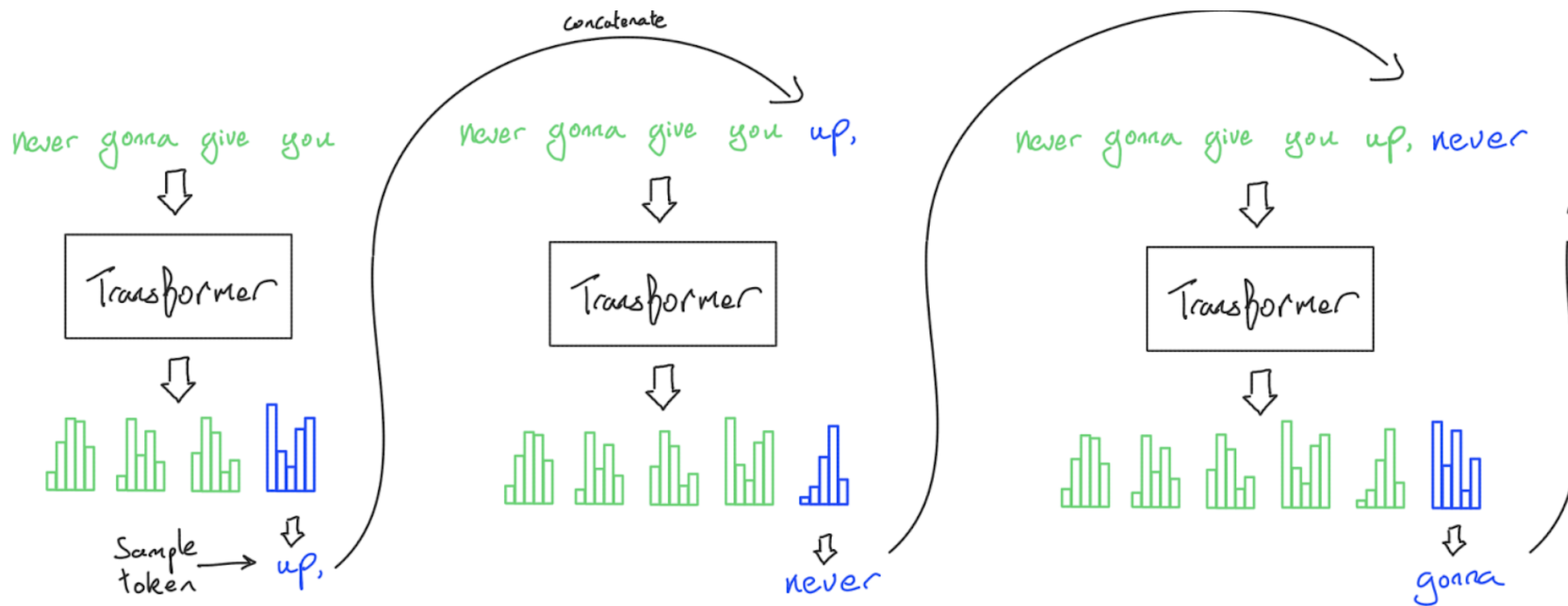
- Log-prob decomposes over tokens:

$$\log \pi_{\theta}(y \mid x) = \sum_{t=1}^T \log \pi_{\theta}(y_t \mid x, y_{<t})$$

 In RL terms:

State: $s_t = (x, y_{<t})$

Action: $a_t = y_t$

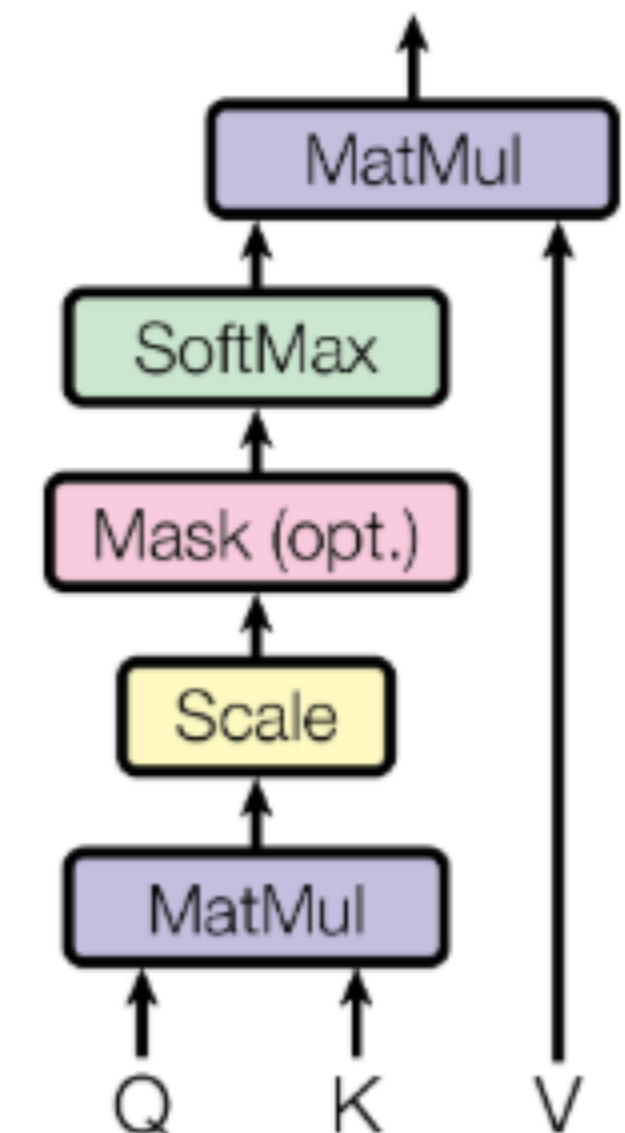


Self-Attention As The Core Primitive

- Input hidden states $H \in \mathbb{R}^{T \times d}$
- Linear projections: $Q = HW_Q$, $K = HW_K$, $V = HW_V$
- Scaled dot-product attention:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Scaled Dot-Product Attention



Causal Masking Gives The Policy Factorization

- **Causal (triangular) mask:** token t attends only to positions $\leq t$
- Masked attention logits:

$$\text{softmax}\left(\frac{QK^\top + M}{\sqrt{d_k}}\right), \quad M_{t,j} = \begin{cases} 0 & j \leq t \\ -\infty & j > t \end{cases}$$

- Consequence: next-token distribution depends only on $\pi_\theta(y_t \mid x, y_{<t})$

Multi-Head Attention

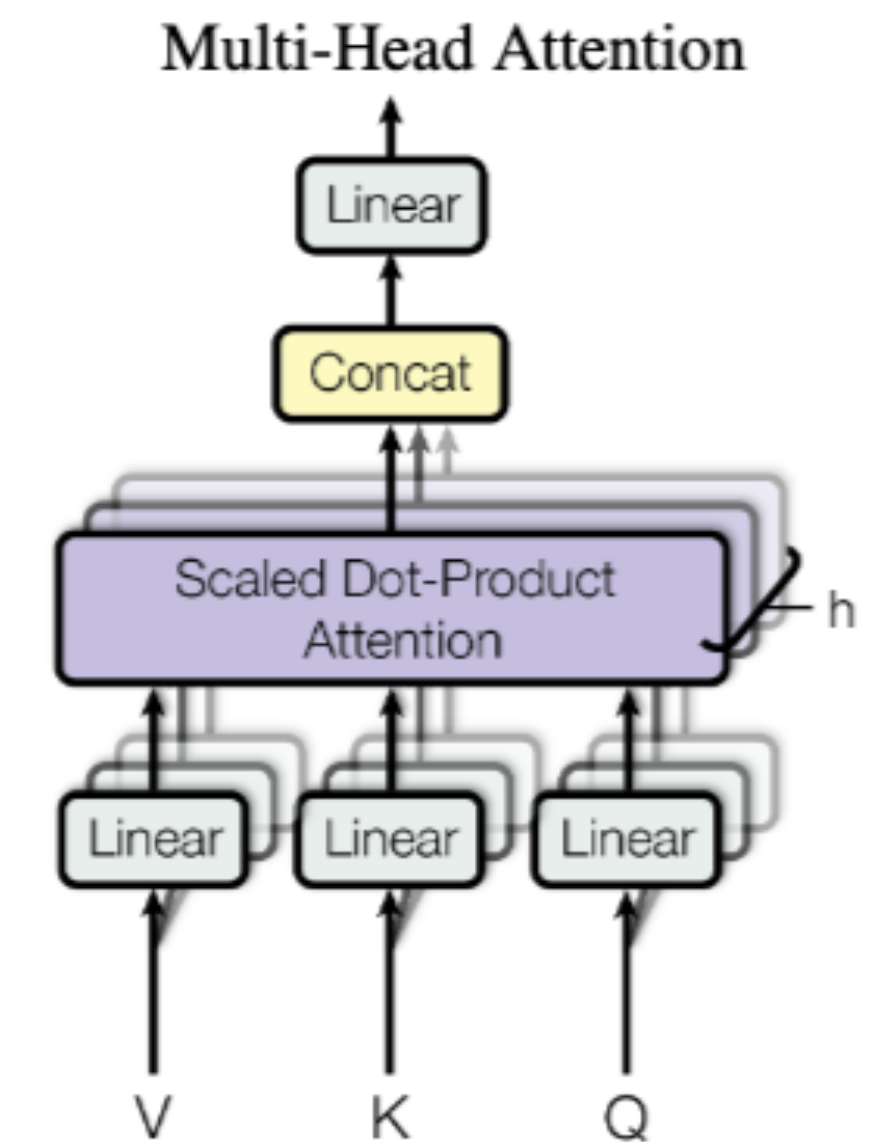
- Multiple heads attend in parallel:

$$\text{head}_i = \text{Attn}(HW_Q^{(i)}, HW_K^{(i)}, HW_V^{(i)})$$

- Concatenate + project:

$$\text{MHA}(H) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$

- Intuition: different heads learn different matching functions over positions/features.



Pretraining Objective (Maximum Likelihood)

- Dataset of text: (x, y) with completion tokens $y = (y_1, \dots, y_T)$
- Teacher-forcing MLE:

$$\max_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}} \left[\sum_{t=1}^T \log \pi_{\theta}(y_t \mid x, y_{<t}) \right]$$

- Equivalent: minimize cross-entropy / negative log-likelihood.

Why Pretraining Is Not Enough

- Pretraining fits **text distribution**: what appears on the internet
- Desired behavior is conditional and normative:
 - follow user intent (helpfulness)
 - avoid unsafe outputs (harmlessness)
 - be truthful / calibrated (honesty)
- Mismatch: “most likely continuation” ≠ “best assistant action”

Alignment Targets As Objectives

- We want behavior that is helpful, honest, and harmless (HHH)
- Model this as reward design (or multi-objective reward):

$$r(x, y) = w_H r_{\text{help}}(x, y) + w_T r_{\text{truth}}(x, y) - w_S r_{\text{harm}}(x, y)$$

- Or as a constraint (common in safety):

$$\max_{\theta} \mathbb{E}[r_{\text{help}}] \quad \text{s.t.} \quad \mathbb{E}[r_{\text{harm}}] \leq \delta$$

- Post-training methods differ in how they optimize, not in what they target

Supervised Fine-Tuning

- Data: instruction prompts x with **demonstration** responses $y^\star$
- Train by teacher forcing (next-token prediction on the demo)
- SFT produces a strong reference policy π_{ref} for later stages

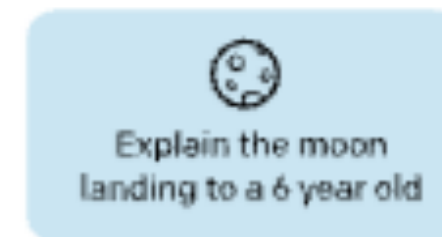
$$\max_{\theta} \mathbb{E}_{(x, y^\star) \sim \mathcal{D}_{\text{demo}}} \left[\sum_{t=1}^T \log \pi_{\theta}(y_t^\star \mid x, y_{<t}^\star) \right]$$

 SFT $\approx$ imitate good behavior; RLHF $\approx$ improve it under preference/safety signals.”

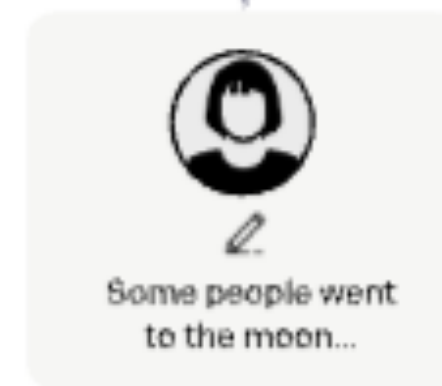
Step 1

**Collect demonstration data,
and train a supervised policy.**

A prompt is
sampled from our
prompt dataset.



A labeler
demonstrates the
desired output
behavior.



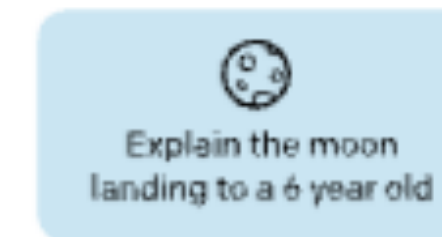
This data is used
to fine-tune GPT-3
with supervised
learning.



Step 2

**Collect comparison data,
and train a reward model.**

A prompt and
several model
outputs are
sampled.



A labeler ranks
the outputs from
best to worst.



This data is used
to train our
reward model.



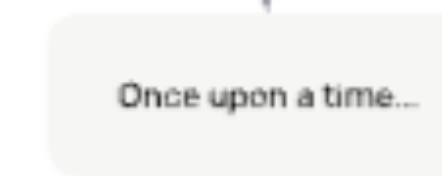
Step 3

**Optimize a policy against
the reward model using
reinforcement learning.**

A new prompt
is sampled from
the dataset.



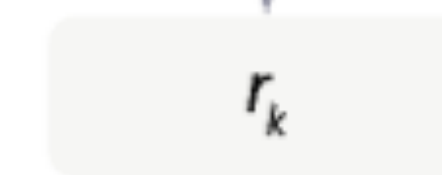
The policy
generates
an output.



The reward model
calculates a
reward for
the output.



The reward is
used to update
the policy
using PPO.



SFT As KL Minimization

- Let $p_{\text{demo}}(y \mid x)$ be the (unknown) demonstration distribution
- SFT maximizes $\mathbb{E}_{p_{\text{demo}}} \log \pi_{\theta}(y \mid x)$
- Equivalent viewpoint: fit $\pi_{\theta}(\cdot \mid x)$ to $p_{\text{demo}}(\cdot \mid x)$

$$\arg \max_{\theta} \mathbb{E}_{(x,y) \sim p_{\text{demo}}} [\log \pi_{\theta}(y \mid x)] = \arg \min_{\theta} \mathbb{E}_x \left[\text{KL} \left(p_{\text{demo}}(\cdot \mid x) \parallel \pi_{\theta}(\cdot \mid x) \right) \right]$$

MLE Equals KL Minimization

- Start from expected log-likelihood: $\mathbb{E}_{y \sim p} [\log \pi_{\theta}(y)]$
- Add and subtract $\log p(y)$

$$\mathbb{E}_p [\log \pi_{\theta}(y)] = \mathbb{E}_p [\log p(y)] - \mathbb{E}_p \left[\log \frac{p(y)}{\pi_{\theta}(y)} \right]$$

- Therefore:

$$\mathbb{E}_p [\log \pi_{\theta}(y)] = -H(p) - \text{KL}(p \parallel \pi_{\theta})$$

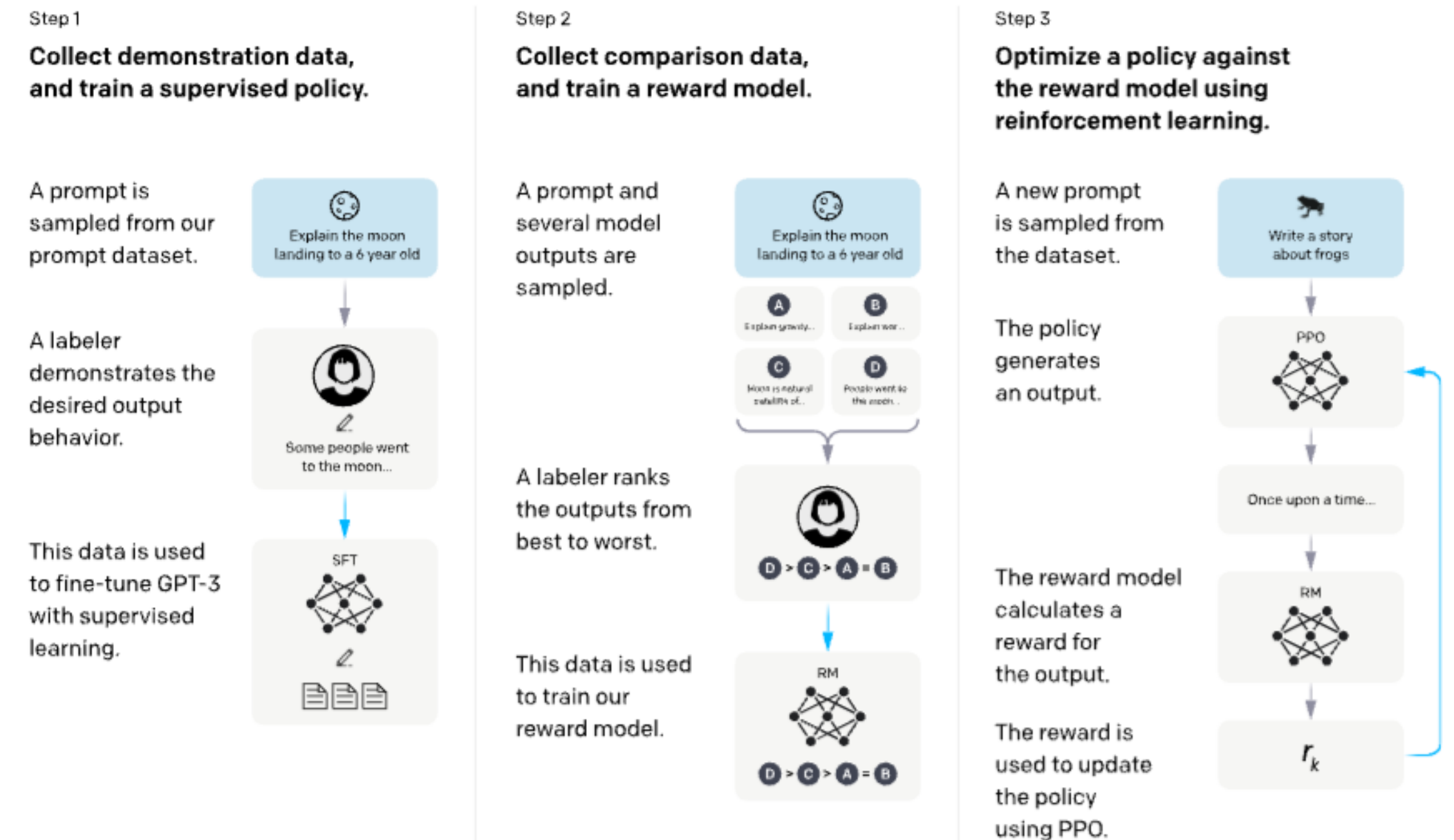
Why SFT Defines π_{ref}

- SFT gives a competent instruction-following baseline
- Later optimization updates π_{θ}
- π_{θ} but keeps it close to π_{ref}
- This prevents:
 - **language drift** (losing fluency / formatting)
 - **reward exploitation** (overfitting the proxy)
 - **instability** (large distribution shift)

$\pi_{\text{ref}} :=$ SFT checkpoint

InstructGPT Pipeline

- **Step 1:** Supervised Fine-Tuning (SFT) → produces π_{ref}
- **Step 2:** Preference rankings → train Reward Model $r_{\phi}(x, y)$
- **Step 3:** Optimize π_{θ} against r_{ϕ} with **KL control** (PPO)



KL-Regularized Post-Training Objective

- Reward signal $r(x, y)$ (from RM / verifier / rubric)
- Reference policy $\pi_{\text{ref}}(\cdot | x)$ (usually SFT)
- Optimize:

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)}[r(x, y)] - \beta \mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)} \left[\log \frac{\pi_{\theta}(y | x)}{\pi_{\text{ref}}(y | x)} \right]$$

Equivalent Constrained Form (Trust Region View)

- Penalized form:

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}}[r(x, y)] - \beta \mathbb{E}_{y \sim \pi_{\theta}} \left[\log \frac{\pi_{\theta}(y | x)}{\pi_{\text{ref}}(y | x)} \right]$$

- Constrained form (same spirit):

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}}[r(x, y)] \quad \text{s.t.} \quad \mathbb{E}_x [\text{KL}(\pi_{\theta}(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x))] \leq \varepsilon$$

- Intuition: “Improve reward, but stay in a KL ball around π_{ref} ”

Proof: Closed-Form Optimal Policy 1/2

Fix a prompt x . Optimize over distributions $\pi(\cdot | x)$ on a finite set of completions $y \in \mathcal{Y}$

$$\max_{\pi} \sum_{y \in \mathcal{Y}} \pi(y | x) r(x, y) - \beta \sum_{y \in \mathcal{Y}} \pi(y | x) \log \frac{\pi(y | x)}{\pi_{\text{ref}}(y | x)}$$

subject to (for every prompt): $\sum_{y \in \mathcal{Y}} \pi(y | x) = 1$ and $\pi(y | x) \geq 0$

Proof: Closed-Form Optimal Policy 1/2

Fix a prompt x . Optimize over distributions $\pi(\cdot | x)$ on a finite set of completions $y \in \mathcal{Y}$

$$\max_{\pi} \sum_{y \in \mathcal{Y}} \pi(y | x) r(x, y) - \beta \sum_{y \in \mathcal{Y}} \pi(y | x) \log \frac{\pi(y | x)}{\pi_{\text{ref}}(y | x)}$$

subject to (for every prompt): $\sum_{y \in \mathcal{Y}} \pi(y | x) = 1$ and $\pi(y | x) \geq 0$

$$\text{Lagrangian: } \mathcal{L}(\pi, \lambda) = \sum_y \pi(y | x) r(x, y) - \beta \sum_y \pi(y | x) \log \frac{\pi(y | x)}{\pi_{\text{ref}}(y | x)} + \lambda \left(\sum_y \pi(y | x) - 1 \right).$$

Proof: Closed-Form Optimal Policy 1/2

Fix a prompt x . Optimize over distributions $\pi(\cdot | x)$ on a finite set of completions $y \in \mathcal{Y}$

$$\max_{\pi} \sum_{y \in \mathcal{Y}} \pi(y | x) r(x, y) - \beta \sum_{y \in \mathcal{Y}} \pi(y | x) \log \frac{\pi(y | x)}{\pi_{\text{ref}}(y | x)}$$

subject to (for every prompt): $\sum_{y \in \mathcal{Y}} \pi(y | x) = 1$ and $\pi(y | x) \geq 0$

$$\text{Lagrangian: } \mathcal{L}(\pi, \lambda) = \sum_y \pi(y | x) r(x, y) - \beta \sum_y \pi(y | x) \log \frac{\pi(y | x)}{\pi_{\text{ref}}(y | x)} + \lambda \left(\sum_y \pi(y | x) - 1 \right).$$

Stationary condition (for each y)

$$\frac{\partial \mathcal{L}}{\partial \pi(y | x)} = r(x, y) - \beta \left(\log \frac{\pi(y | x)}{\pi_{\text{ref}}(y | x)} + 1 \right) + \lambda = 0.$$

Proof: Closed-Form Optimal Policy 2/2

Rearrange:

$$\log \frac{\pi(y | x)}{\pi_{\text{ref}}(y | x)} = \frac{r(x, y)}{\beta} + \frac{\lambda}{\beta} - 1.$$

Exponentiate:

$$\pi(y | x) = \pi_{\text{ref}}(y | x) \exp\left(\frac{r(x, y)}{\beta}\right) \cdot \exp\left(\frac{\lambda}{\beta} - 1\right).$$

Let $Z(x) := \sum_y \pi_{\text{ref}}(y | x) \exp(r(x, y)/\beta)$. Enforcing normalization gives

$$\exp\left(\frac{\lambda}{\beta} - 1\right) = \frac{1}{Z(x)}.$$

Conclusion

$$\pi^*(y | x) = \frac{\pi_{\text{ref}}(y | x) \exp(r(x, y)/\beta)}{\sum_{y'} \pi_{\text{ref}}(y' | x) \exp(r(x, y')/\beta)}$$

Reference = Imitation Prior; Reward = Tilt

- Closed-form optimum:

$$\pi^*(y \mid x) = \frac{\pi_{\text{ref}}(y \mid x) \exp(r(x, y)/\beta)}{Z(x)}$$

- Log form (useful intuition):

$$\log \pi^*(y \mid x) = \log \pi_{\text{ref}}(y \mid x) + \frac{r(x, y)}{\beta} - \log Z(x)$$

- Effect of β
 - $\beta \downarrow$: stronger tilt toward high reward (more drift)
 - $\beta \uparrow$: weaker tilt (stay close to π_{ref})

Token-Level Vs Sequence-Level KL

- Sequence KL (per prompt)

$$\text{KL}(\pi_{\theta}(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x)) = \mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)} \left[\log \frac{\pi_{\theta}(y | x)}{\pi_{\text{ref}}(y | x)} \right]$$

- Autoregressive factorization:

$$\log \frac{\pi_{\theta}(y | x)}{\pi_{\text{ref}}(y | x)} = \sum_{t=1}^T \log \frac{\pi_{\theta}(y_t | x, y_{<t})}{\pi_{\text{ref}}(y_t | x, y_{<t})}$$

- Therefore (practical lens):

$$\text{KL}(\text{sequence}) = \sum_{t=1}^T \mathbb{E}_{y \sim \pi_{\theta}} \left[\log \frac{\pi_{\theta}(y_t | x, y_{<t})}{\pi_{\text{ref}}(y_t | x, y_{<t})} \right]$$

(expectation is under rollouts from π_{θ}).

Why KL Is Non-Negotiable

- Post-training uses proxy rewards (RM / heuristics / verifiers) → imperfect signals
- Without a KL anchor, optimization can exploit proxy errors:

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}}[r(x, y)] \quad \Rightarrow \quad \text{seek } y \text{ that hacks } r$$

- KL penalty enforces conservative updates:

$$\max_{\theta} \mathbb{E}[r] - \beta \mathbb{E} \left[\log \frac{\pi_{\theta}}{\pi_{\text{ref}}} \right]$$

- Typical failure modes when KL is too weak:
 - **Reward hacking** (proxy exploitation)
 - **Distribution shift** (off-policy RM errors worsen)
 - **Language/style drift** (verbosity, weird formatting, repetition)

Lecture B

PPO Motivation: Unconstrained Updates Can Blow Up

- Objective we would like to improve (sequence-level):

$$J(\theta) = \mathbb{E}_{y \sim \pi_{\theta}(\cdot|x)}[R(x, y)]$$

- First-order “surrogate” around π_{old} using importance sampling:

$$J(\theta) \approx \mathbb{E}_{y \sim \pi_{\text{old}}} \left[\frac{\pi_{\theta}(y | x)}{\pi_{\text{old}}(y | x)} R(x, y) \right]$$

- Problem: the ratio $\pi_{\theta}/\pi_{\text{old}}$ can become large $\Rightarrow$ unstable jumps
- Fix: enforce conservative updates (trust-region / clipping)

Importance Ratio For Token Actions

- State/action for an autoregressive LM: $s_t = (x, y_{<t})$, $a_t = y_t$
- Token-level importance ratio:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)} = \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{old}}(y_t \mid x, y_{<t})}$$

- Sequence-level ratio factorizes:

$$\frac{\pi_\theta(y \mid x)}{\pi_{\text{old}}(y \mid x)} = \prod_{t=1}^T r_t(\theta), \quad \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{old}}(y \mid x)} = \sum_{t=1}^T \log r_t(\theta)$$

PPO Clipped Surrogate Objective

- Define the clipped ratio:

$$\text{clip}(r_t, 1 - \epsilon, 1 + \epsilon)$$

- PPO surrogate (per token):

$$L^{\text{clip}}(\theta) = \mathbb{E} \left[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t \right) \right]$$

- Total objective sums over tokens (and batch):

$$\max_{\theta} \mathbb{E} \left[\sum_{t=1}^T L_t^{\text{clip}}(\theta) \right]$$

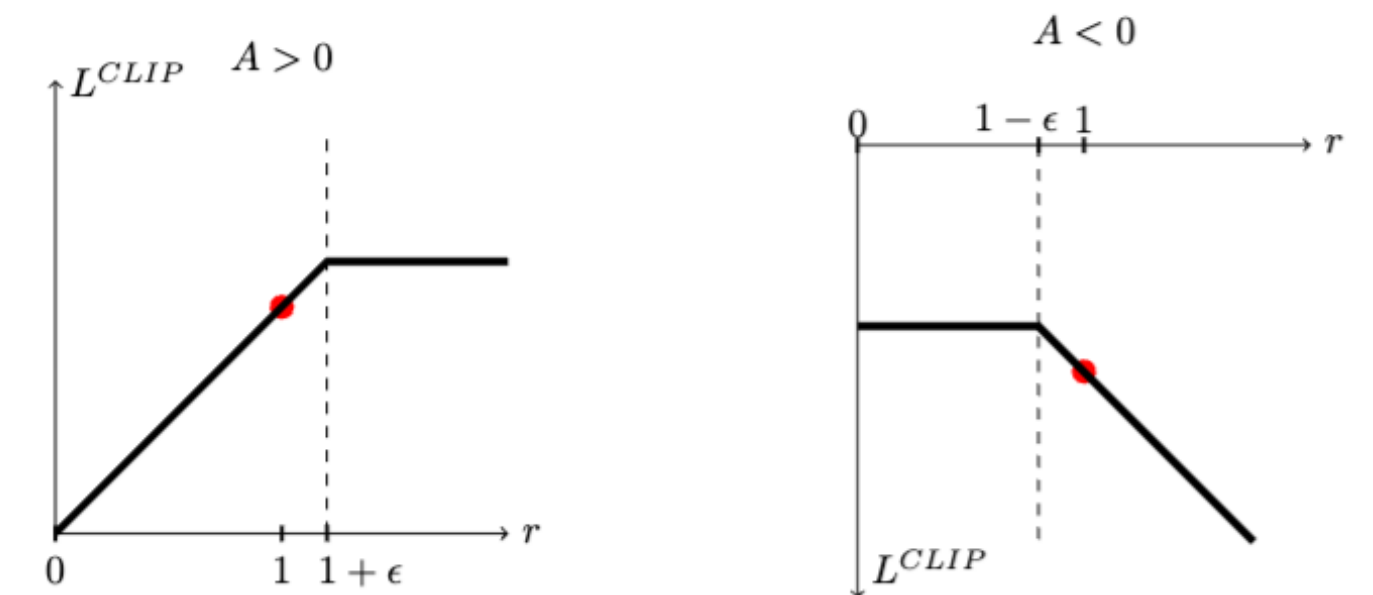


Figure 1: Plots showing one term (i.e., a single timestep) of the surrogate function L^{CLIP} as a function of the probability ratio r , for positive advantages (left) and negative advantages (right). The red circle on each plot shows the starting point for the optimization, i.e., $r = 1$. Note that L^{CLIP} sums many of these terms.

RLHF Reward: Score Minus KL

- Reward model score (from Lecture 7):

$$r_{\text{RM}}(x, y) \in \mathbb{R}$$

- KL penalty to reference policy:

$$\text{KL}(\pi_{\theta}(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x)) = \mathbb{E}_{y \sim \pi_{\theta}} \left[\log \frac{\pi_{\theta}(y | x)}{\pi_{\text{ref}}(y | x)} \right]$$

- Define the RLHF return:

$$R(x, y) = r_{\text{RM}}(x, y) - \beta \text{KL}(\pi_{\theta}(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x))$$

Advantage Estimation In RLHF

- Sequence return (single scalar):

$$R(x, y) = r_{\text{RM}}(x, y) - \beta \text{KL}(\pi_{\theta} || \pi_{\text{ref}})$$

- Token-level credit via advantage:

$$A_t = \hat{Q}(s_t, a_t) - \hat{V}(s_t)$$

- In the simplest sequence-level setting (common in LLM RLHF):

$$\hat{Q}(s_t, a_t) \approx R(x, y), \quad A_t \approx R(x, y) - b(x)$$

where $b(x)$ is a baseline (e.g., value model or mean return for prompt).

PPO: What Can Go Wrong

- **Instability:** high-variance returns + on-policy sampling → noisy gradients
- **Over-optimization of proxy:** RM exploitation; quality may degrade
- **KL collapse / explosion:** too small β → drift; too large β → no learning
- **Engineering cost:** rollouts + careful tuning

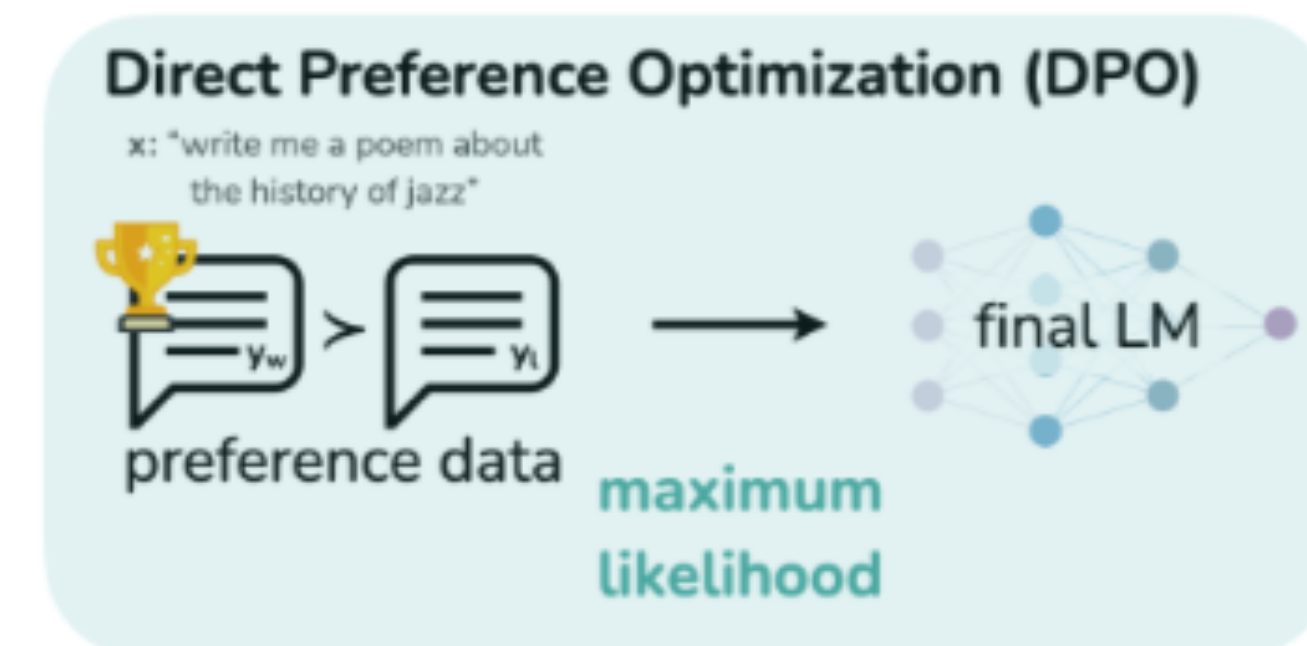
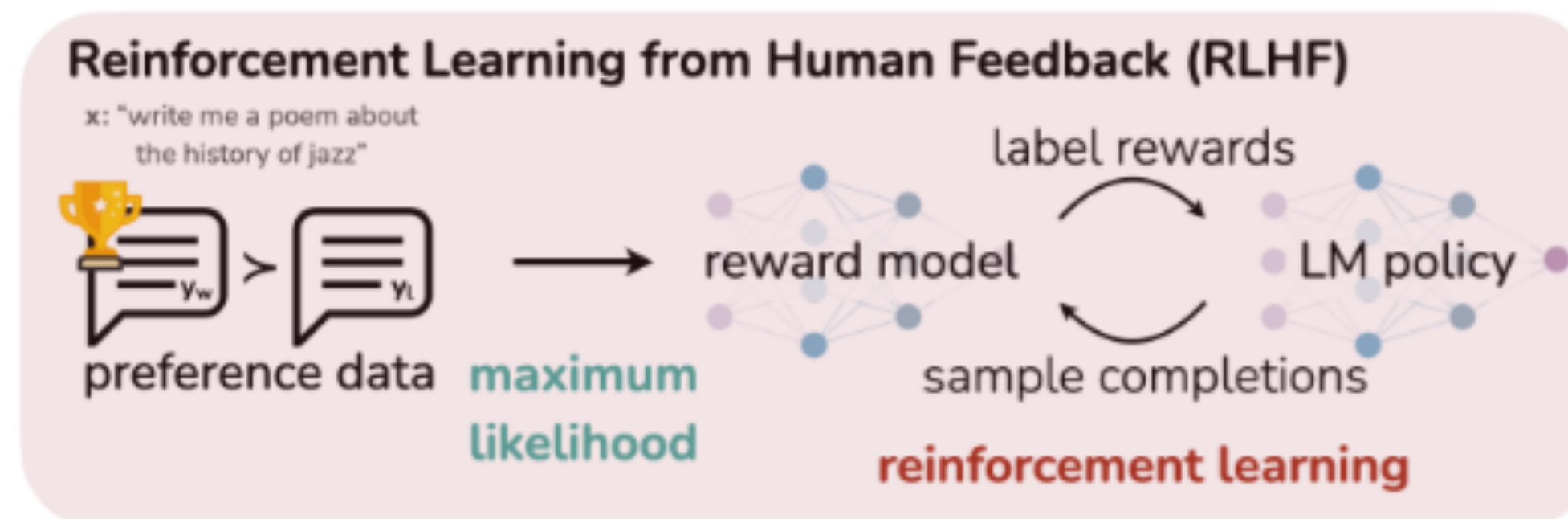
Direct Preference Optimization (DPO)

DPO: Optimize From Preference Pairs

- Data: (x, y^+, y^-) , $y^+ \succ y^-$
- Choose a fixed reference π_{ref} (usually SFT)
- Train π_{θ} , by maximizing a preference log-likelihood

$$\max_{\theta} \mathbb{E} [\log P_{\theta}(y^+ \succ y^- \mid x)]$$

- Key move: parameterize $P_{\theta}(\cdot)$ using **policy log-prob ratios** (next slides)



Key Identity From The KL-Regularized Optimum

- From the previous theorem, for fixed x

$$\pi^*(y \mid x) \propto \pi_{\text{ref}}(y \mid x) \exp(r(x, y)/\beta)$$

- Taking logs and rearranging gives:

$$r(x, y) = \beta \left(\log \pi^*(y \mid x) - \log \pi_{\text{ref}}(y \mid x) \right) + C(x)$$

where

$$C(x) = \beta \log Z(x) \quad \text{and} \quad Z(x) = \sum_{y'} \pi_{\text{ref}}(y' \mid x) \exp(r(x, y')/\beta).$$

Bradley–Terry Likelihood In Policy Log-Ratios

- Bradley–Terry model (Lecture 7):

$$P(y^+ \succ y^- \mid x) = \sigma(r(x, y^+) - r(x, y^-))$$

Substitute the difference form (dropping $C(x)$) : $r(x, y^+) - r(x, y^-) = \beta(\Delta_\theta - \Delta_{\text{ref}})$

where

$$\Delta_\theta = \log \pi_\theta(y^+ \mid x) - \log \pi_\theta(y^- \mid x), \quad \Delta_{\text{ref}} = \log \pi_{\text{ref}}(y^+ \mid x) - \log \pi_{\text{ref}}(y^- \mid x).$$

- Therefore:

$$P_\theta(y^+ \succ y^- \mid x) = \sigma(\beta(\Delta_\theta - \Delta_{\text{ref}}))$$

DPO Loss

- Given preference pairs (x, y^+, y^-) , define

$$u_{\theta}(x, y^+, y^-) = \beta \left[(\log \pi_{\theta}(y^+ | x) - \log \pi_{\theta}(y^- | x)) - (\log \pi_{\text{ref}}(y^+ | x) - \log \pi_{\text{ref}}(y^- | x)) \right].$$

- Then the DPO objective is maximum likelihood:

$$\max_{\theta} \mathbb{E} [\log \sigma(u_{\theta})]$$

- Equivalently, minimize the loss:

$$\mathcal{L}_{\text{DPO}}(\theta) = - \mathbb{E}_{(x, y^+, y^-)} [\log \sigma(u_{\theta}(x, y^+, y^-))]$$

MLE On Preferences $\Rightarrow$ DPO Loss

- Model:

$$P_{\theta}(y^{+} \succ y^{-} \mid x) = \sigma(u_{\theta}(x, y^{+}, y^{-}))$$

- Log-likelihood over dataset $\mathcal{D}$:

$$\ell(\theta) = \sum_{(x, y^{+}, y^{-}) \in \mathcal{D}} \log \sigma(u_{\theta}(x, y^{+}, y^{-}))$$

- Negative log-likelihood:

$$-\ell(\theta) = \sum_{(x, y^{+}, y^{-}) \in \mathcal{D}} -\log \sigma(u_{\theta}(x, y^{+}, y^{-})) \equiv |\mathcal{D}| \cdot \mathcal{L}_{\text{DPO}}(\theta)$$

What DPO Is Doing (Gradient View)

- For one pair, loss is $\ell(\theta) = -\log \sigma(u_\theta)$. Then,

$$\frac{\partial \ell}{\partial u_\theta} = -(1 - \sigma(u_\theta)).$$

And,

$$\frac{\partial u_\theta}{\partial \theta} = \beta \left(\nabla_\theta \log \pi_\theta(y^+ | x) - \nabla_\theta \log \pi_\theta(y^- | x) \right).$$

So

$$\nabla_\theta \ell(\theta) = -\beta(1 - \sigma(u_\theta)) \left(\nabla_\theta \log \pi_\theta(y^+ | x) - \nabla_\theta \log \pi_\theta(y^- | x) \right).$$

Role Of β in DPO

- Appears in the preference model:

$$P_{\theta}(y^{+} \succ y^{-} \mid x) = \sigma\left(\beta(\Delta_{\theta} - \Delta_{\text{ref}})\right)$$

- Appears in gradient magnitude:

$$\|\nabla_{\theta}\ell\| \propto \beta(1 - \sigma(u_{\theta}))$$

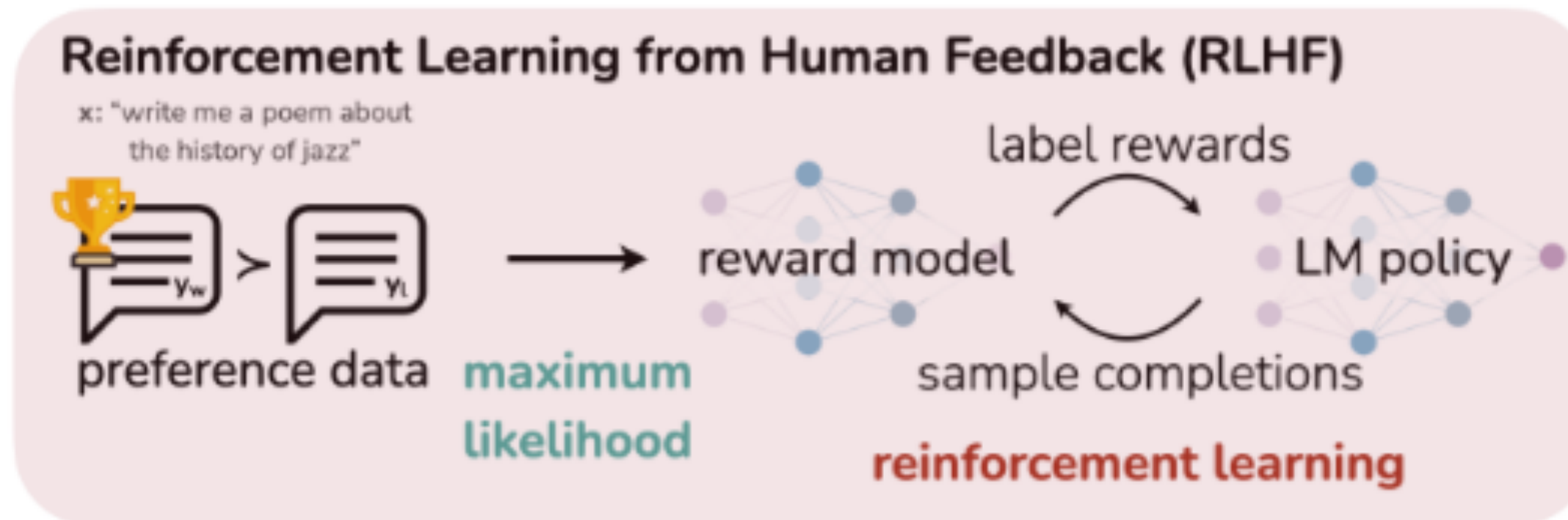
- Connection to KL-regularized optimum:

$$\pi^{*}(y \mid x) \propto \pi_{\text{ref}}(y \mid x)\exp(r(x, y)/\beta),$$

so β controls how strongly reward reshapes the reference distribution.

DPO Pipeline

- Inputs: preference pairs (x, y^+, y^-)
- Compute: $\log \pi_\theta(y^\pm | x)$ and $\log \pi_{\text{ref}}(y^\pm | x)$
- Optimize: $\mathcal{L}_{\text{DPO}}(\theta) = -\log \sigma(u_\theta)$



DPO Practical Checklist (What To Monitor)

- Monitor preference accuracy / win-rate on held-out pairs:

$$\mathbb{P}(u_{\theta}(x, y^+, y^-) > 0)$$

- Monitor KL drift from reference (token KL estimate):

$$\widehat{\text{KL}} \approx \frac{1}{T} \sum_t (\log \pi_{\theta} - \log \pi_{\text{ref}})$$

- Watch length/verbosity drift (often correlates with log-prob shifts)
- Reference choice: π_{ref} fixed SFT vs moving reference (changes the target)